
2-Developer Parallel Development Plan

Reconciliation & Reporting Tool

1. Recommended Split

Do **not** split randomly screen by screen.

For this system, the cleanest split is:

Developer 1: Platform + External Operations

Owns everything related to:

- Authentication
- Layout / shared UI shell
- User management
- Admin module
- OpCo module
- Partner module
- Report upload / report viewing
- Notification & reminder settings
- Basic dashboard structure for Admin / OpCo / Partner

Developer 2: Core Financial Operations

Owns everything related to:

- Client reconciliation module
- Invoice management
- Collection tracking
- Partner payment tracking
- Search / export
- Reconciliation engine logic
- Client dashboard

- Reporting / analytics widgets
- Audit logs

This is the cleanest split because:

- Dev 1 handles **user-facing intake and setup side**
- Dev 2 handles **core business processing side**

That reduces overlap massively.

2. Final Ownership Breakdown

Developer 1 Ownership

Modules

- Authentication & Access Control
- Role-based routing
- User Management
- Admin Settings
- OpCo Report Module
- Partner Report Module
- Partner Invoice Upload screen
- Notification & Reminder Module
- Master Data basic setup:
 - OpCos
 - Partners
 - Services
- Shared app shell and navigation
- Non-client dashboards

Screens

- Login
- Forgot Password
- Change Password
- Admin Dashboard
- User Management
- Reminder Settings
- Notification Settings

- OpCo Dashboard
- Partner Dashboard
- Upload Report
- View Reports
- Upload Partner Invoice
- View Own Invoices

Backend ownership

- auth
 - users
 - masters
 - reports
 - notifications
 - reminder settings
-

Developer 2 Ownership

Modules

- Client Dashboard
- Reconciliation Workbench
- Report Review / Accept / Reject
- OpCo Invoice Generation
- Invoice Review / Status Update
- Collection Tracking
- Partner Invoice Processing
- Partner Payment Tracking
- Search / Filter / Export
- Audit Logs
- Adhoc Reporting
- Core reconciliation rules and workflows

Screens

- Client Dashboard
- View All Reports
- Review Report
- View All Invoices
- Update Invoice Status
- Reconciliation Workbench
- Collection Tracking

- Partner Payment Processing
- Search Results
- Export screens if separate
- Audit Logs
- Financial reports / summaries

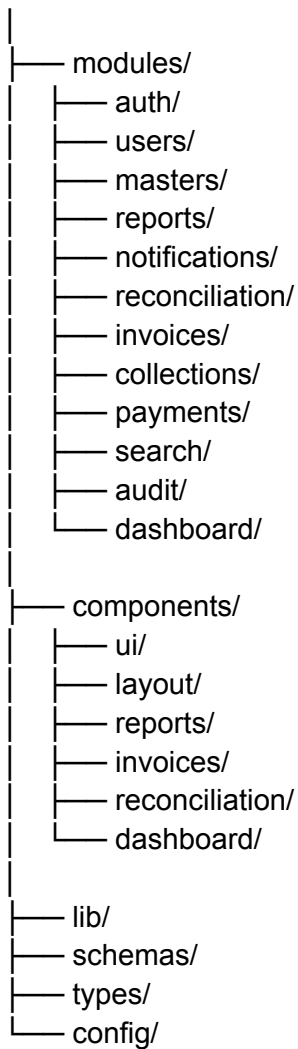
Backend ownership

- reconciliation
- invoices
- collections
- payments
- search
- export
- audit
- reporting

3. Folder Structure to Prevent Conflicts

Use this structure so each developer owns separate areas.

```
src/
├── app/
│   ├── (auth)/
│   ├── (dashboard)/
│   │   ├── admin/
│   │   ├── opco/
│   │   ├── partner/
│   │   └── client/
│   └── api/
│       ├── auth/
│       ├── users/
│       ├── masters/
│       ├── reports/
│       ├── notifications/
│       ├── reconciliation/
│       ├── invoices/
│       ├── collections/
│       ├── payments/
│       ├── search/
│       └── audit/
```



4. Exact File/Folder Ownership

Developer 1 should own

app/(auth)/**
app/(dashboard)/admin/**
app/(dashboard)/opco/**
app/(dashboard)/partner/**
app/api/auth/**
app/api/users/**
app/api/masters/**
app/api/reports/**

app/api/notifications/**

modules/auth/**

modules/users/**

modules/masters/**

modules/reports/**

modules/notifications/**

components/reports/**

components/layout/**

Developer 2 should own

app/(dashboard)/client/**

app/api/reconciliation/**

app/api/invoices/**

app/api/collections/**

app/api/payments/**

app/api/search/**

app/api/audit/**

modules/reconciliation/**

modules/invoices/**

modules/collections/**

modules/payments/**

modules/search/**

modules/audit/**

modules/dashboard/client/**

components/reconciliation/**

components/invoices/**

components/dashboard/client/**

Shared but controlled

These files should be touched by **one person first only**, then treated as stable shared base:

app/layout.tsx

components/ui/**

lib/db.ts

lib/auth.ts

types/**

schemas/common/**
config/**

Best approach:

- **Developer 1 builds shared shell first**
 - **Developer 2 starts only after shared contracts are frozen**
 - after that, shared files should be edited only through discussion
-

5. Best GitHub Collaboration Strategy

Branches

Use:

- `main` → protected
- `dev` → integration branch
- `dev1/<feature-name>`
- `dev2/<feature-name>`

Example:

- `dev1/auth-module`
- `dev1/opco-report-upload`
- `dev2/reconciliation-workbench`
- `dev2/invoice-management`

Do **not** work directly on `main`

Do **not** even work directly on `dev`

6. Merge Flow

Use this exact flow:

Step 1

Both developers pull latest `dev`

Step 2

Each works only on their feature branch

Step 3

Create PR into `dev`

Step 4

Other developer reviews PR

Step 5

Merge into `dev`

Step 6

Only after testing, merge `dev` into `main`

This avoids chaos.

7. Shared Contract First, Then Coding

Before both start full development, define these things together first:

Must freeze before parallel work:

- route structure
- role names
- folder structure
- database table names
- API naming conventions
- response structure
- shared types
- status names
- enum values

Because most merge conflicts happen when:

- one dev says `invoiceStatus`
- other says `status`
- one says `partner_id`
- other says `partnerId`

That destroys speed.

8. API Contract Rule

Each developer should not wait for the other to finish backend.

Instead define API contracts first.

Example:

Reports API owned by Dev 1

- POST `/api/reports/upload`
- GET `/api/reports`
- GET `/api/reports/:id`
- PATCH `/api/reports/:id/review`

Reconciliation/Invoice APIs owned by Dev 2

- POST `/api/reconciliation/run`
- GET `/api/reconciliation/results`
- POST `/api/invoices/generate`
- PATCH `/api/invoices/:id/status`
- POST `/api/collections`
- POST `/api/payments`

Once request/response contracts are frozen, each dev can build independently.

9. Database Ownership Rule

This is critical.

Do **not** let both devs edit schema randomly.

Best method:

Option A

One developer owns full schema migration file
recommended

Better choice here:

Let **Developer 2** own financial tables

Let **Developer 1** own auth/master/user/report tables

But only **one person** should update final Prisma schema file at a time after agreement.

Safer split:

- Developer 1 drafts:
 - users
 - roles
 - opcos
 - partners
 - services
 - reports
 - notification_settings
- Developer 2 drafts:
 - reconciliations
 - reconciliation_items
 - invoices
 - collections
 - payments
 - audit_logs

Then combine once, carefully.

10. UI Conflict Prevention

Do not let both devs build UI differently.

Set these shared rules first:

- one design system
- one spacing system
- one button style
- one table component
- one form component
- one modal pattern
- one page header style

Best approach:

- Dev 1 builds shared layout, navbar, sidebar, table wrapper, form wrapper, buttons, inputs
- Dev 2 reuses those only

That prevents visual inconsistency and code conflicts.

11. Equal Workload Reality Check

This split is close to equal, but not naturally equal unless managed properly.

Developer 2 has heavier logic

because reconciliation, invoices, payments, collections are more complex.

So to balance properly:

Give Developer 1 these additional responsibilities:

- full master data management
- full notification engine UI/settings
- full report upload/review structure
- admin dashboard widgets
- reusable data table/filter system

That makes workload much more balanced.

12. Recommended Delivery Sequence

Phase 1: Foundation

Developer 1

- app shell
- auth
- layout
- sidebar
- shared UI
- role routing
- user management

Developer 2

- define reconciliation/invoice data models
- create core service interfaces
- prepare financial module structures

Phase 2: Independent Module Development

Developer 1

- reports
- opco/partner dashboards
- notifications
- reminder settings
- masters

Developer 2

- reconciliation
- invoice management
- collections
- payments
- client dashboard
- audit/search/export

Phase 3: Integration

- connect report acceptance to reconciliation

- connect finalized reconciliation to invoice generation
 - connect collection received to partner invoice eligibility
 - connect payments to dashboard/reporting
-

13. PR Rules to Avoid Conflicts

Use these rules:

Every PR must:

- target one module only
- avoid unrelated refactors
- not modify shared files unless necessary
- include updated types if API changed
- include note on impacted routes/modules

Never in same PR:

- auth + reconciliation
- reports + payments
- UI redesign + business logic
- schema changes + many unrelated screens

Keep PRs small.

14. Best Daily Coordination Rule

Have one shared file:

`PROJECT_OWNERSHIP.md`

Inside it define:

- who owns which module
- who owns which routes
- who owns which APIs
- who owns which tables
- which shared files are frozen

This single file will save you from stupid conflicts.

15. My Recommended Final Split

Developer 1

Platform / Admin / External Submission Lead

- Auth
- Users
- Masters
- Admin
- OpCo
- Partner
- Reports
- Notifications
- Shared UI shell

Developer 2

Financial Core / Client Operations Lead

- Reconciliation
- Invoice lifecycle
- Collections
- Payments
- Search/export
- Audit
- Client dashboard
- Reporting analytics

That is the most practical split for this project.

16. One Important Warning

Do not split by:

- frontend vs backend
- screen by screen randomly
- both devs touching all modules together

That always creates:

- merge conflicts
- duplicate logic
- inconsistent APIs
- broken ownership

For your project, **domain-based split** is the correct one.